EdgeFogSim: Edge, Fog and Cloud Task Placement Simulator

ARM 305 – Cloud, Dev, Edge and Fog Computing · Mini Project Report

Team: Moksh (06719051624) · Bhavya Bisht (05019051624) · Dhruv Sharma (05719051624) · Bhuwan Chugh (04219051624)

Repository: github.com/moksh-m9u/Cloud-Dew-Edge-network-simulation-engine

1. Objective

Edge and fog computing move processing closer to where data is produced, so urgent jobs can be answered in milliseconds, while large jobs still go to the cloud. Deciding where each job should run is a trade-off: nearby nodes are fast to reach but small, the cloud is huge but far away.

EdgeFogSim is a locally runnable Python simulator of that decision. It accepts computing tasks, places each on an Edge, Fog or Cloud node with a rule-based algorithm, simulates the resources it uses through virtual-machine records, keeps tenants' records apart, and reports placement decisions, latency, utilisation, success and failure. No real cloud, VMs or hypervisor are needed. Every number the simulator uses is a setting that can be changed without touching the code.

2. Architecture

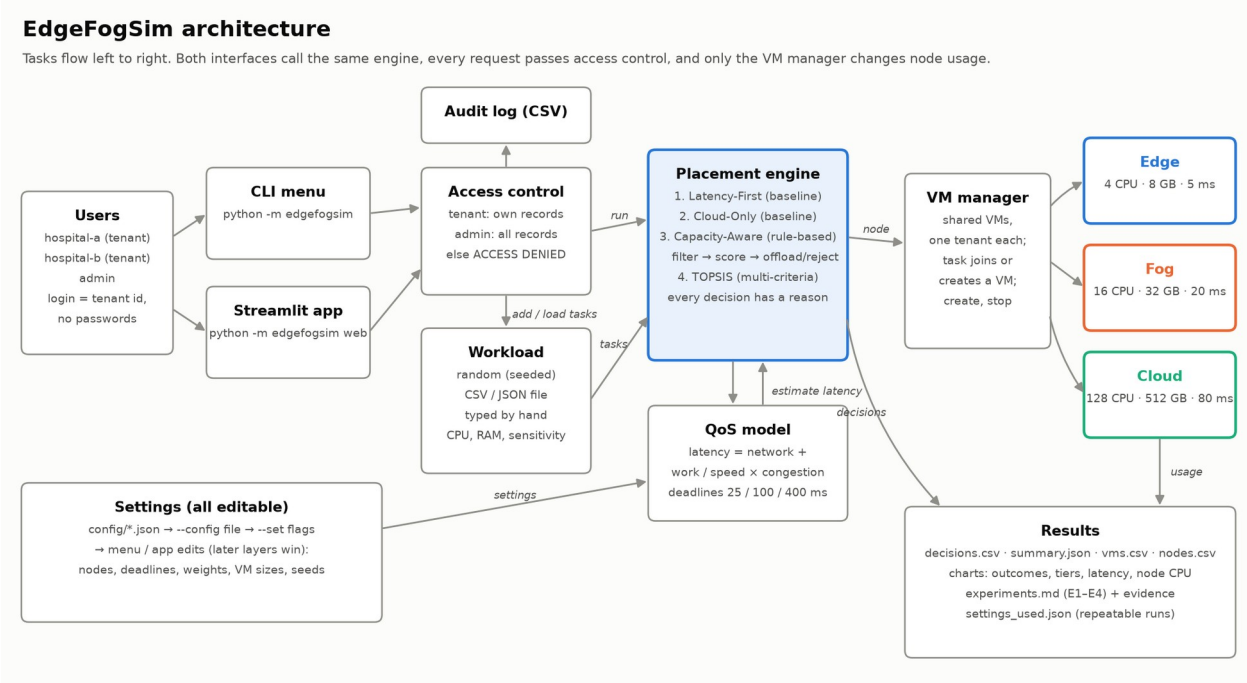


Figure 1. EdgeFogSim architecture.

- **Users and interfaces.** Tenants (hospital-a, hospital-b) and an admin log in by choosing a tenant id; no passwords exist. The same engine is driven from a command-line menu or a Streamlit web app.
- **Access control.** Every request passes one rule: a tenant may see and act on their own records, the admin on all. Anything else is refused with ACCESS DENIED and written to a CSV audit log.
- **Workload.** Tasks have CPU, memory, a latency sensitivity (high, medium, low) and an amount of work. They are generated with a fixed seed, loaded from CSV/JSON, or typed in.
- **Placement engine and QoS model.** A policy picks a node for each task using the latency model below; every decision records its reason.
- **VM manager and nodes.** Tasks run inside tenant-owned VMs on the chosen Edge, Fog or Cloud node. This is the only component that changes node usage.
- **Results.** Decisions, summaries, VMs and nodes are saved as CSV/JSON with PNG charts, plus the settings used so any run can be repeated.

3. Placement algorithm

Default nodes: Edge 4 CPU, 8 GB, 5 ms away, speed 0.6; Fog 16 CPU, 32 GB, 20 ms away, speed 1; Cloud 128 CPU, 512 GB, 80 ms away, speed 2. Each task's deadline depends on its sensitivity: high 25 ms, medium 100 ms, low 400 ms.

Latency model

latency = network latency + (work ÷ node speed) × congestion, congestion = 1 ÷ (1 − CPU load after placement), with load capped at 95%.

Congestion is a queueing (M/M/1-style) approximation: 1 on an idle node, 2 at half load, 10 at 90% load. A task is successful if it is placed and its latency is within its deadline.

Capacity-Aware: the rule-based algorithm

1. Order the tiers by the task's sensitivity: high → Edge, Fog, Cloud; medium → Fog, Edge, Cloud; low → Cloud, Fog, Edge.
2. Capacity filter: the node must be able to host the task (in a VM of the task's tenant with room, or in a new VM on unreserved capacity) and its CPU load must stay at or below 85%.
3. Deadline filter: the estimated latency must be within the task's deadline.
4. Score each remaining node: 0.5 × tier preference + 0.3 × time to spare before the deadline + 0.2 × CPU left free.
5. Place the task on the highest score. If that node is not in the task's first-choice tier, record it as an offload, with the reason.
6. If no node passes, retry once allowing nodes to be fully loaded (soft overload).
7. If still none passes, reject the task and record which check failed on each node.

Two baselines are compared with it: Latency-First (nearest node with room, ignoring deadlines) and Cloud-Only. A fourth policy, TOPSIS, ranks the feasible nodes by closeness to an ideal node on latency, free CPU and free memory, a multi-criteria method used in fog node selection research.

Task	Needs	Sensitivity	Decision	Latency	Why
T01	2 CPU, 4 work	high	edge-1	18.3 ms	best score on edge-1 (edge, preference 1 of 3); est. 18.3 ms ≤ 25 ms deadline
T02	2 CPU, 4 work	high	fog-1 (offload)	24.6 ms	offloaded to fog: edge-1 would be 100% busy (limit 85%); best score on fog-1 (fog, preference 2 of 3); est. 24.6 ms ≤ 25 ms deadline
T03	8 CPU, 40 work	low	cloud-1	101.3 ms	best score on cloud-1 (cloud, preference 1 of 3); est. 101.3 ms ≤ 400 ms deadline
T04	20 CPU, 4 work	high	rejected	-	no node passes: edge-1 not enough CPU for a new VM (needs 20, 2 unreserved); fog-1 not enough CPU for a new VM (needs 20, 12 unreserved); cloud-1 too slow (est. 82.6 ms > 25 ms ...

Table 1. Capacity-Aware on the first four sample tasks, starting from empty nodes.

4. Implementation

Module	Role
config	Layered, validated settings: defaults → config/*.json → --config file → flags → menu edits
models	Node, Task, Tenant and VM dataclasses
workload	Seeded generator, CSV/JSON task files, typed tasks, row-level validation
qos / policies	Latency model; Latency-First, Cloud-Only, Capacity-Aware and TOPSIS
vm_manager / simulator	Shared tenant VMs (join, create, stop); runs a workload, records decisions
security	Access rule, ACCESS DENIED, CSV audit log
results / charts	pandas tables, CSV/JSON export, matplotlib charts
experiments	E1–E4 with expected outcomes checked against actual numbers
cli / web	Command-line menu (rich) and Streamlit app; no simulation logic of their own

Table 2. Python modules in the edgefogsim package.

Virtualisation. A VM is a slice of one node owned by one tenant. A placed task joins one of its tenant's VMs on the chosen node if it has room; otherwise a new VM is created, sized to the tier default (edge 2 CPU / 4 GB, fog 4 CPU / 8 GB, cloud 8 CPU / 16 GB) or to the task. Nodes therefore track reserved capacity (held by VMs) and load (used by tasks); load drives latency. VMs can be created ahead of time and stopped, which frees their slice.

Technology. Python 3.14, pandas for result tables, matplotlib for charts, rich for the terminal menu and Streamlit for the web app; settings are JSON. Quality is checked by 160 automated pytest tests mapped to the acceptance criteria in docs/SPEC.md, including tests that drive both interfaces and one that requires a docstring on every function.

5. Experiments and results

All four experiments run with python run_experiments.py. Each states its expected outcome before running and checks it against the actual numbers; 4 of 4 matched (19 of 19 checks). Full evidence is in docs/evidence/experiments.

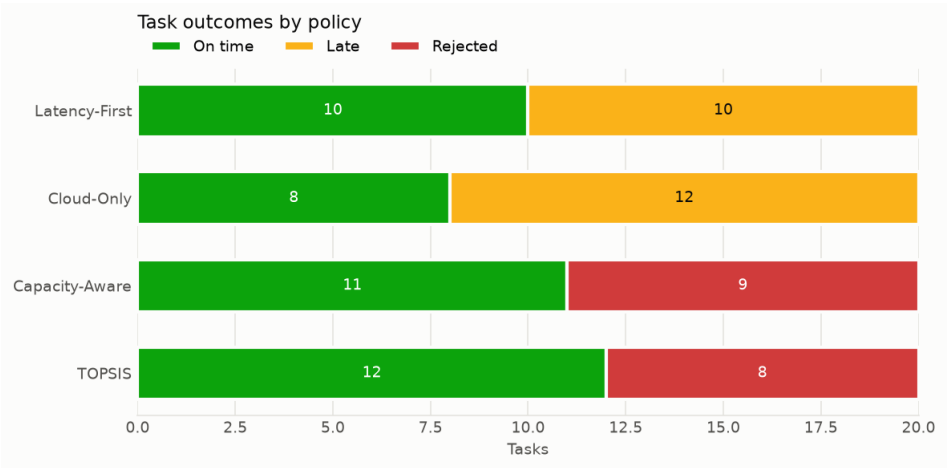
E1: Policy comparison

Setup. Workload: 20 generated tasks, seed 42 (11 high, 5 medium, 4 low sensitivity). Policies: Latency-First, Cloud-Only, Capacity-Aware, TOPSIS. Each policy starts from empty nodes, so the comparison is fair.

Expected. The smart policies (Capacity-Aware, TOPSIS) place no task late, and get more tasks on time than both baselines (Latency-First, Cloud-Only), which ignore deadlines.

Policy	On time	Late	Rejected	Offloaded	Success	Avg ms	P95 ms
Latency-First	10	10	0	0	50%	73.6	114.8
Cloud-Only	8	12	0	16	40%	91.1	118.5
Capacity-Aware	11	0	9	3	55%	70.9	120.3
TOPSIS	12	0	8	4	60%	67.8	120.3

Actual. 4 of 4 checks held: Capacity-Aware places no task late (0 late); Capacity-Aware gets more tasks on time (11) than the best baseline (10); TOPSIS places no task late (0 late); TOPSIS gets more tasks on time (12) than the best baseline (10).



E1: Task outcomes by policy.

E2: Overload

Setup. Workload: 10 generated tasks, seed 7 (10 high sensitivity). Before the burst: edge-1 already 90% busy. Policies: Latency-First, Capacity-Aware.

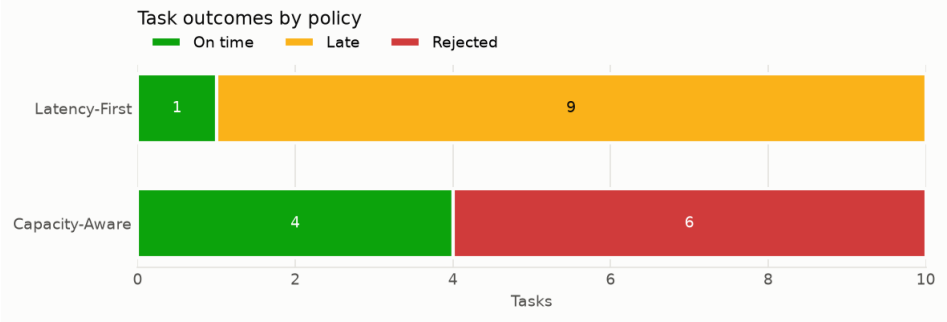
Expected. Capacity-Aware moves tasks that don't fit on the busy Edge node to another node (offloaded) and rejects, with a reason, tasks that no node can finish in time. It runs nothing late, never allocates more than a node has, and does better than Latency-First, which keeps placing tasks where they will be late.

Policy	On time	Late	Rejected	Offloaded	Success	Avg ms	P95 ms
Latency-First	1	9	0	0	10%	46.4	82.1
Capacity-Aware	4	0	6	4	40%	24.1	24.9

Offload, T02 → fog-1: offloaded to fog: edge-1 not enough CPU for a new VM (needs 2, 0.4 unreserved); best score on fog-1 (fog, preference 2 of 3); est. 24.6 ms ≤ 25 ms deadline; scores: fog-1 0.43; runs in new VM vm-0001 (4 CPU / 8 GB)

Rejection, T01: no node passes: edge-1 not enough CPU for a new VM (needs 2, 0.4 unreserved); fog-1 too slow (est. 25.3 ms > 25 ms deadline); cloud-1 too slow (est. 82.5 ms > 25 ms deadline)

Actual. 5 of 5 checks held: Capacity-Aware offloads at least one task (4 offloaded); Capacity-Aware rejects tasks no node can serve in time (6 rejected, each with a reason); Capacity-Aware places no task late (0 late); No node is ever allocated beyond its capacity; Capacity-Aware gets at least as many tasks on time (4) as Latency-First (1, with 9 late).



E2: Task outcomes under overload.

E3: Unauthorised tenant access

Setup. Workload: 10 tasks from tasks_sample.csv. Placed with Capacity-Aware. Logged in as hospital-b, who tries to read hospital-a's task T01, read its VM vm-0001, and stop that VM. Then the admin reads the same records.

Expected. Every attempt by hospital-b is refused with ACCESS DENIED, returns nothing, and adds one line to the audit log. hospital-a's VM keeps running. The admin can read both records.

timestamp	actor	target_tenant	resource	outcome
2026-11-02T23:18:21	hospital-b	hospital-a	task T01	DENIED
2026-11-02T23:18:21	hospital-b	hospital-a	VM vm-0001	DENIED
2026-11-02T23:18:21	hospital-b	hospital-a	VM vm-0001	DENIED

Audit log

hospital-b → read task T01: ACCESS DENIED: hospital-b may not access task T01 (belongs to hospital-a)

hospital-b → read VM vm-0001: ACCESS DENIED: hospital-b may not access VM vm-0001 (belongs to hospital-a)

hospital-b → stop VM vm-0001: ACCESS DENIED: hospital-b may not access VM vm-0001 (belongs to hospital-a)

admin → read task T01 and VM vm-0001: allowed

Actual. 4 of 4 checks held: All 3 attempts by hospital-b were refused (3 refused); The audit log has one line per refusal (3 lines); hospital-a's VM vm-0001 is still running; The admin can read both records.

E4: Node capacity and latency change

Setup. Workload: 20 generated tasks, seed 42 (11 high, 5 medium, 4 low sensitivity). Policy: Capacity-Aware. Each change is compared with the original nodes, one at a time.

Expected. More CPU on a node lets it take more tasks, so at least as many finish on time and no more are rejected. More network latency on a node makes every task placed there slower; tasks whose deadline no longer allows that node move elsewhere or are rejected, while low-sensitivity tasks still finish on time.

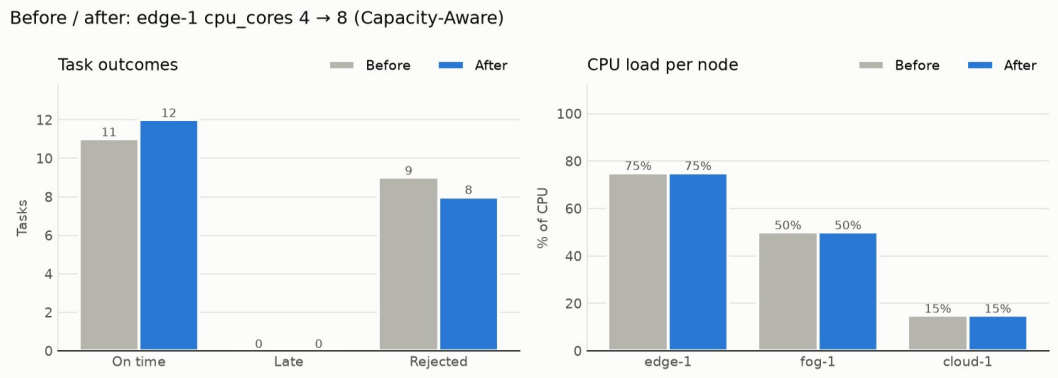
Metric	Before	After
on time	11	12
late	0	0
rejected	9	8
offloaded	3	3
avg latency ms	70.9	65.3
p95 latency ms	120.3	120.3
tasks on edge	1	2
tasks on fog	4	4
tasks on cloud	6	6

Change a: edge-1 cpu_cores 4 → 8

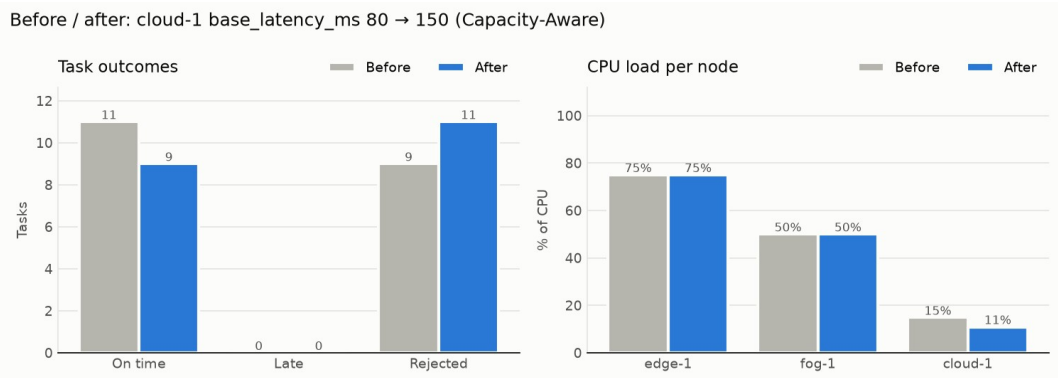
Metric	Before	After
on time	11	9
late	0	0
rejected	9	11
offloaded	3	1
avg latency ms	70.9	96.6
p95 latency ms	120.3	189.6
tasks on edge	1	1
tasks on fog	4	4
tasks on cloud	6	4

Change b: cloud-1 base_latency_ms 80 → 150

Actual. 6 of 6 checks held: edge-1 takes at least as many tasks (1 → 2); At least as many tasks on time (11 → 12); No more tasks rejected (9 → 8); Tasks on cloud-1 are slower on average (104 → 179 ms); Low-sensitivity tasks all still finish on time (4 / 4); No task runs late (late 0 → 0).



E4: Before and after: edge-1 cpu_cores 4 → 8.



E4: Before and after: cloud-1 base_latency_ms 80 → 150.

6. Discussion

- **Deadline-aware placement pays off.** Capacity-Aware ran 0 tasks late and TOPSIS 0, against 10 for Latency-First and 12 for Cloud-Only. They reject what cannot be finished in time instead of running it late.

- **Rejection is the price.** The smart policies refuse some urgent tasks when Edge is full and Fog is too slow. In a real deployment this is the signal to add Edge capacity, which E4a shows helps.
- **Utilisation is not the same as quality.** Latency-First keeps Edge and Fog fully loaded, but the congestion makes tasks late; a lower, steadier load gives better service.
- **Isolation holds.** Every cross-tenant attempt in E3 was refused and logged, while the admin could still read everything.

7. Limitations and future work

- **Simulated time.** Tasks are placed one after another and keep running until their VM is stopped; there is no task duration, arrival time or network jitter.
- **Analytical latency.** Latency comes from a formula, not measurement; it captures distance, speed and congestion but not bandwidth or data size.
- **Greedy, per-task decisions.** Policies decide one task at a time. Batch optimisers such as genetic algorithms or PSO, and a learning policy (tabular Q-learning, planned as an optional extension), could find better global placements.
- **TOPSIS rank reversal.** Closeness scores can reorder when the set of feasible nodes changes.
- **No live migration or real authentication.** VMs are never moved, and login is choosing a tenant id, as the brief requires.

8. Team contributions

Member	Enrollment number	Contribution
Moksh	06719051624	Placement policies, simulator, experiments
Bhavya Bisht	05019051624	VM manager and security, Streamlit app
Dhruv Sharma	05719051624	Results export and charts, testing
Bhuwan Chugh	04219051624	Report, slides, architecture diagram